

Theory of the dashboard

wd-magnetizada

What each tab computes: equations, code
and the decisions that cost real time

July 23, 2026

Magnetized white dwarf with a mixed interior field (poloidal + toroidal)
Technical reference document — not an introduction to stellar physics or MHD

Contents

Table of symbols and units	3
1 Common theoretical core	4
1.1 Equation of state	4
1.2 Self-gravity	5
1.3 Magnetic field and the flux function	5
1.4 Grad-Shafranov	6
1.5 Bernoulli and the parametrization	7
1.6 The SCF loop	8
1.7 Virial and energy diagnostics	9
1.8 The two Bt/Bp ratios	10
1.9 The toroidal field and the twisted torus	10
1.10 Time scales and units	11
2 Tab 1 — Equilibrium	13
3 Tab 2 — Sweep	14
4 Tab 3 — Export	15
5 Tab 4 — Runs	16
6 Known limitations	16
7 Open questions	17
8 References	18

This document explains what the dashboard computes: the equations behind each number and each figure, and the code that implements them. It is not an introduction to stellar physics or MHD — it assumes the reader already knows that. What this document does is bridge the theory (section 1) and the real code (`scf/`, `dashboard/`).

See `plano_wd_magnetizada.md` for the full project plan (decisions D1–D6, architecture, phases). This document is about the *implemented physics*, not the work plan.

Table of symbols and units

Symbol	Meaning	Unit (CGS)	Name in code
r, θ	spherical coordinates	cm, rad	<code>r, theta</code>
ϖ	cylindrical radius, $\varpi = r \sin \theta$	cm	<code>omega</code> (see note)
z	height, $z = r \cos \theta$	cm	<code>z</code> (only in <code>plots.py</code>)
ρ	mass density	g cm^{-3}	<code>rho</code>
x	normalized Fermi momentum, $x = p_F/(m_e c)$	dimensionless	<code>x</code>
P	pressure	dyn cm^{-2}	<code>pressure(x)</code>
H	specific enthalpy	erg g^{-1}	<code>H</code>
Φ	gravitational potential	erg g^{-1}	<code>Phi</code>
A_φ	φ component of the vector potential	G cm	<code>A_phi</code>
u	flux function, $u = \varpi A_\varphi$	G cm^3	<code>u</code>
B_r, B_θ, B_φ	magnetic field components	G (gauss)	<code>Br, Bth/Btheta, Bphi</code>
$f(u)$	poloidal current function, $f(u) = k_0$	$\text{g}^{-1/2} \text{cm}^{1/2} \text{s}^{-1}$	<code>k0</code>
$M(u)$	poloidal Bernoulli potential	erg g^{-1}	<code>M_u</code> (local in <code>scf.py</code>)
$\beta(u)$	toroidal current function, $\beta = \varpi B_\varphi$	G cm	not computed separately
ζ	amplitude of the imposed toroidal field	—	<code>zeta</code>
m	exponent of the imposed toroidal field	integer ≥ 1	<code>m_tor</code>
u_c	value of u on the last closed line	G cm^3	<code>u_c</code>
C	Bernoulli constant	erg g^{-1}	<code>C</code>
ρ_c	central density (input parameter)	g cm^{-3}	<code>rho_c</code>
μ_e	mean molecular weight per electron, $Y_e = 1/\mu_e$	dimensionless	<code>mu_e</code>
M	total stellar mass	g (displayed in $M_\odot$)	<code>M</code> (collides with $M(u)$ — see note)
R_{eq}, R_{pol}	equatorial and polar radii (where $H = 0$)	cm (displayed in km)	<code>R_eq, R_pol</code>
W	gravitational energy	erg	<code>W</code>
Π	internal energy, $\int P dV$	erg	<code>Pi</code>
$E_{pol}, E_{tor}, \mathcal{M}$	magnetic energies	erg	<code>E_pol, E_tor, E_mag</code>
VE	virial error	dimensionless	<code>VE</code>
G	gravitational constant	$6.674 \times 10^{-8} \text{ cgs}$	<code>G_CONST</code>
A, B	EOS constants	see §1.1	<code>A_CONST, B_of_mu_e()</code>
l	degree of the Legendre expansion	integer $\geq 0/\geq 1$	<code>l</code>
t_{dyn}, v_A, t_{Alf}	derived scales	s, cm/s, s	<code>t_dyn, v_A, t_alf</code>

Notation notes:

- The code uses the variable **omega** for ϖ (cylindrical radius), **not** for angular velocity — there is no rotation in this project (D3, static star). It is a naming choice specific to this code; whenever you read **omega** in `scf.py/gradshafranov.py`, it always means $\varpi = r \sin \theta$.
- M is used in the source theory for two different objects: the total stellar mass and the Bernoulli potential $M(u)$ (§1.5). The code resolves the collision by naming the mass **M** (returned by `scf.total_mass()`) and the potential **M_u** (a local variable inside `hachisu_scf()`, never exposed outside the function). In this document, whenever there is risk of ambiguity, the text spells out “mass” or “Bernoulli potential” in full.
- **Bth** appears as **Btheta** in `diagnostics.py` (function parameter) and **Bth** on the dashboard pages (local variable) — same object, two names.

1 Common theoretical core

1.1 Equation of state

Fully degenerate electron gas at $T = 0$:

$$P = A \left[x(2x^2 - 3)(x^2 + 1)^{1/2} + 3 \sinh^{-1} x \right] \quad (1)$$

$$\rho = Bx^3 \quad (2)$$

$$x \equiv \frac{p_F}{m_e c} \quad (3)$$

$$A = 6.01 \times 10^{22} \text{ dyn cm}^{-2} \quad (4)$$

$$B = \frac{9.82 \times 10^5}{Y_e} \text{ g cm}^{-3} \quad (Y_e = 0.5 \Rightarrow B \approx 1.96 \times 10^6) \quad (5)$$

A is a fixed physical constant (a combination of m_e , c , $\hbar$); B depends on composition only through Y_e (electrons per baryon). The dashboard parametrizes by $\mu_e = 1/Y_e$ instead of Y_e directly.

→ `scf/eos.py :: pressure()`

(`P(x)`), `density()` ($\rho(x)$), `B_of_mu_e()` ($B(\mu_e)$)

Specific enthalpy, analytic — this is what makes the SCF possible:

$$H = \int \frac{dP}{\rho} = \frac{8A}{B} \left[\sqrt{1 + x^2} - 1 \right] \quad (6)$$

→ `scf/eos.py :: enthalpy()`

Inverse, applied at every SCF iteration:

$$x = \sqrt{\left(1 + \frac{HB}{8A}\right)^2 - 1} \quad (H \leq 0 \Rightarrow \rho = 0) \quad (7)$$

→ `scf/eos.py :: x_of_enthalpy(), density_of_enthalpy()`

The -1 term normalizes $H = 0$ at the surface (the $\rho = 0$ boundary of the star).

Effective polytropic index. Near the surface $x \ll 1$ gives $\rho \propto H^{3/2}$, i.e. $n = 3/2$. In the relativistic core $x \gg 1$ gives $\rho \propto H^3$, i.e. $n = 3$. The EOS slides between the two, and $n = 3$ is the marginal case where the mass becomes independent of the central density — the origin of the Chandrasekhar limit. This fact is the direct cause of the parametrization decision in §1.4/§1.5.

Sound speed. Added after the original theoretical spine (it was not in the plan), because it is needed for the amplitude of the perturbation exported in Tab 3:

$$c_s = \sqrt{\frac{dP}{d\rho}}, \quad \frac{dP}{dx} = \frac{8Ax^4}{\sqrt{1+x^2}} \quad (\text{analytic derivative of } P(x)) \quad (8)$$

→ `scf/eos.py :: sound_speed()`

, validated against a finite difference in `scf/tests/test_sound_speed.py`

1.2 Self-gravity

$$\nabla^2 \Phi = 4\pi G \rho \quad (9)$$

Solved by expansion in Legendre polynomials. With $\rho(r, \theta) = \sum_l \rho_l(r) P_l(\cos \theta)$:

$$\Phi_l(r) = -\frac{4\pi G}{2l+1} \left[r^{-(l+1)} \int_0^r \rho_l(r') r'^{l+2} dr' + r^l \int_r^\infty \rho_l(r') r'^{1-l} dr' \right] \quad (10)$$

The weights r'^{l+2} and r'^{1-l} carry the r'^2 of the volume element.

→ `scf/poisson.py :: solve_poisson()`

(exact implementation of this formula, D_l/E_l computed by cumulative trapezoidal sums); `legendre_matrix()` computes $P_l(\cos \theta)$.

Validated against the closed-form solution of the uniform sphere and Newton's shell theorem in `scf/tests/test_poisson.py`.

$G = 6.674 \times 10^{-8} \text{ cm}^3 \text{ g}^{-1} \text{ s}^{-2}$ — a fixed physical constant (rounded CODATA reference value), not a project parameter.

→ `scf/poisson.py :: G_CONST`

, repeated in `dashboard/units.py :: G_CONST` (same value, two modules, because `poisson.py` cannot depend on the dashboard — R1).

1.3 Magnetic field and the flux function

With $\varpi = r \sin \theta$, the flux function is defined as

$$u = \varpi A_\varphi \quad (11)$$

and the axisymmetric field splits into poloidal plus toroidal:

$$\mathbf{B} = \frac{1}{\varpi} [\nabla u \times \hat{e}_\varphi + \beta(u) \hat{e}_\varphi] \quad (12)$$

In spherical components:

$$B_r = \frac{1}{r^2 \sin \theta} \frac{\partial u}{\partial \theta} \quad (13)$$

$$B_\theta = -\frac{1}{r \sin \theta} \frac{\partial u}{\partial r} \quad (14)$$

$$B_\varphi = \frac{\beta(u)}{\varpi} \quad (15)$$

→ `scf/diagnostics.py :: poloidal_field()`

implements B_r , B_θ (via finite differences of u , protected against the coordinate singularity at $\varpi \rightarrow 0$). B_φ is not computed from an intermediate $\beta(u)$; it comes directly out of `scf/toroidal.py :: impose_toroidal()`, which already implements the chosen functional form for β (§1.9) substituted into the formula above.

The general form $\mathbf{B} = \frac{1}{\varpi}[\nabla u \times \hat{e}_\varphi + \beta \hat{e}_\varphi]$ does not appear anywhere in the code as a vector identity — the code goes straight to the three scalar components.

Geometric interpretation: the contours of u are the poloidal field lines. That’s why Tab 1 plots them (§2).

1.4 Grad-Shafranov

$$\Delta^* u = -4\pi\varpi^2 \rho f(u) - \beta\beta'(u) \quad (16)$$

$$\Delta^* = \frac{\partial^2}{\partial \varpi^2} - \frac{1}{\varpi} \frac{\partial}{\partial \varpi} + \frac{\partial^2}{\partial z^2} \quad (17)$$

$f(u) = dM/du$ is the current function generating the poloidal field; $\beta(u)$ generates the toroidal field. The simplest choice, and the one used here, is $f(u) = k_0$ constant (Lander & Jones 2009). Since the toroidal field is imposed after the poloidal field has converged (§1.9, D6) and does not enter the GS equation actually solved by the SCF, the term $-\beta\beta'(u)$ **does not appear in the source the code actually assembles** — the implemented source is only $-4\pi\varpi^2 \rho k_0$.

→ `scf/gradshafranov.py :: solve_gradshafranov()`

— source passed by `scf/scf.py :: hachisu_scf()` as `source = -4*np.pi*omega2*rho*k0`.

The current density corresponding to this source is

$$J_\varphi = c\rho\varpi f(u) \quad (18)$$

This formula **does not correspond to any function in the code** — J_φ is never computed as a named quantity. It was used only analytically, during the debugging that found the bug described in the box below, to assemble the right-hand side of the integral Ampère’s-law test (derivative-free tests, further below).

Solution by associated-Legendre expansion. The operator Δ^* separates into $P_l^1(\cos \theta)$ functions, **not** $P_l(\cos \theta)$. The radial structure of the Green’s function is analogous to that of Poisson, but the integral weights **are not the same**:

	inner weight	outer weight
Poisson (for Φ)	r'^{l+2}	r'^{1-l}
Grad-Shafranov (for u)	r'^{l+1}	r'^{-l}

→ `scf/gradshafranov.py :: solve_gradshafranov()`

In the lines where D_l/E_l are assembled with $r^{** (l + 1)}$ and $r^{** (-l)}$.

Why it's like this — the Green's-function exponents

The difference of one power comes from the ϖ weight of the GS source compared to the r'^2 of the volume element in Poisson, and depends on whether the equation is solved for u or for A_φ . **This was a real bug in this project:** an earlier version of `gradshafranov.py` used r'^{l+2} and r'^{1-l} (mistakenly copied from the structure of `poisson.py`, which has one extra power of r because of the substitution $\chi = r\Phi_l$ used in reducing the scalar Laplacian — Δ^* does not need that substitution). The bug inflated the poloidal field by a factor of $\sim 7000\times$ and the magnetic energy by $\sim 5 \times 10^7$.

The bug survived piece-by-piece validation — including a “by-hand” closed form — because that closed form was built by reusing the same indicial equation (r^{l+1} , r^{-l} for the homogeneous solutions, which **were correct**) and therefore inherited the same normalization of the internal integrals, which was wrong. Only two tests that use neither the Green's function nor a second derivative — consistency via Ampère's law — revealed the fixed factor. See the full note (with the reasoning chain) at the top of `scf/gradshafranov.py`.

Operational lesson: “solve with the formula, check against the same formula” is not independent validation, even across different files/functions, if both inherit the same normalization from a shared derivation.

Derivative-free normalization tests — the defense against this class of error:

$$\text{Flux: } u(\varpi, z) = \int_0^\varpi B_z(\varpi', z) \varpi' d\varpi' \quad (19)$$

$$\text{Ampère: } \oint_C \mathbf{B} \cdot d\mathbf{l} = 4\pi k_0 \int_S \rho \varpi dA \quad (20)$$

- **Ampère:** implemented and in the repository

→ `scf/tests/test_gradshafranov.py :: test_ampere_law()`

Uses a synthetic case with a pure $l = 1$ -mode source (not the full EOS/SCF), computes $\oint \mathbf{B} \cdot d\mathbf{l}$ on a rectangular loop in (r, θ) and compares it against $-\iint [\text{source}/\sin\theta] dr d\theta$ (an equivalent form derived from Ampère's law for this geometry). Closes to $\sim 2\%$.

- **Flux:** *proposed* during debugging (see the conversation history that fixed the bug) but **not implemented as a separate test** — the Ampère test alone already revealed and confirmed the bug fix, so the flux consistency test was never actually written. Recorded as a gap in §7.

1.5 Bernoulli and the parametrization

$$H + \Phi - M(u) = C, \quad M(u) = \int f(u) du \quad (21)$$

With $f = k_0$ constant, $M(u) = k_0 u$.

The integration constant is fixed **at the center**:

$$C = H(\rho_c) + \Phi_c - M(u_c) \quad (22)$$

where Φ_c , u_c here denote the values at the center ($r = 0$) — **note:** this use of u_c (value of u at the center) is different from the u_c used in §1.9 (value of u on the last closed line, at the

surface). They are the same symbol for two different evaluation points; the code never needs both at once, but the reader should note the collision. Since $\varpi = r \sin \theta$ vanishes at the center for any θ , $u_c(\text{center}) = 0$ always and the term $M(u_c)$ disappears — it stays in the formula for generality, not because it contributes.

→ scf/scf.py :: hachisu_scf()

— line C = H_c + Phi[0, 0] - M_u[0, 0].

Why it's like this — the (ρ_c, k_0) parametrization

The classical Hachisu recipe fixes C by imposing $H = 0$ at two surface points (pole and equator). This is ill-posed for this EOS: since $n \rightarrow 3$ in the core (§1.1), the mass becomes nearly independent of the central density and the radius plunges, so solving for C from the radius inverts a nearly-singular map. Tested and confirmed in this project: the Picard iteration under that recipe is linearly unstable across the whole range $\rho_c = 10^6\text{--}10^{10}$ g/cm³, with or without sub-relaxation.

The adopted parametrization is by (ρ_c, k_0) , two independent inputs, with no surface condition — C is fixed by a local condition (H at the center), not a global one (an integral over the whole mass). This change alone solved the instability: geometric convergence to machine precision in ~ 12 iterations, versus exponential divergence of the old recipe.

Caveat: in the strong-field regime, when the density peak migrates away from the center, this anchor also loses the physical meaning that makes it stable — see §6.

1.6 The SCF loop

1. $\rho \leftarrow$ initial guess (spherical $n = 3$ polytrope)
2. $A_\varphi \leftarrow 0$
3. **repeat:**
 4. $\Phi \leftarrow \text{Poisson}(\rho)$
 5. $A_\varphi \leftarrow \text{GradShafranov}(\rho, f)$
 6. $u \leftarrow \varpi A_\varphi$; $M \leftarrow \int f du$
 7. $C \leftarrow H(\rho_c) + \Phi_c - M(u_c)$
 8. $H \leftarrow C - \Phi + M(u)$
 9. $\rho_{\text{new}} \leftarrow \text{EOS}^{-1}(H)$ $[H \leq 0 \Rightarrow \rho = 0]$
 10. $\rho \leftarrow (1 - \omega)\rho + \omega\rho_{\text{new}}$ $\omega \approx 0.3$
11. **until** $\max |\Delta\rho|/\rho_c < \text{tol}$

→ scf/scf.py :: hachisu_scf()

implements steps 1, 3–9 and 11 literally (step 2 is implicit: u starts at zero before the first iteration).

Discrepancy with the real code, step 10: the implementation **does not do sub-relaxation**.

The corresponding line in `hachisu_scf()` is `rho = rho_new` — a direct replacement, equivalent to $\omega = 1$, not $\omega \approx 0.3$. There is no ω parameter in the function signature. This is not an oversight: it is a direct consequence of the change described in the §1.5 box. The old recipe (two surface conditions) required sub-relaxation to try to stabilize an iteration that was unstable regardless; the (ρ_c, k_0) parametrization converges by direct replacement because the instability

that sub-relaxation was trying to mask was removed at the root. See the project note at the top of `scf/scf.py` for the full history (including tests showing that sub-relaxing the old recipe did not fix the problem).

The convergence criterion (`tol`) is a numerical, not physical, parameter — the dashboard exposes values from 10^{-4} to 10^{-8} (Tab 1), with 10^{-6} as the default.

1.7 Virial and energy diagnostics

$$W = \frac{1}{2} \int \rho \Phi dV \quad (\text{gravitational, negative}) \quad (23)$$

$$\Pi = \int P dV \quad (\text{internal}) \quad (24)$$

$$E_{pol} = \int \frac{B_r^2 + B_\theta^2}{8\pi} dV \quad (25)$$

$$E_{tor} = \int \frac{B_\varphi^2}{8\pi} dV \quad (26)$$

$$\mathcal{M} = E_{pol} + E_{tor} \quad (27)$$

→ `scf/diagnostics.py :: gravitational_energy()`

(`W`), `pressure_integral()` (`Π`), `magnetic_energies()` (`Epol`, `Etor`, `ℳ` — called `E_mag` in the code). All use `volume_integral()` as their base ($dV = r^2 \sin \theta dr d\theta d\varphi$, φ integrated analytically to 2π).

Scalar virial theorem for a static configuration:

$$W + 3\Pi + \mathcal{M} = 0 \quad (28)$$

Virial error, used as a quality gate:

$$\text{VE} = \frac{|W + 3\Pi + \mathcal{M}|}{|W|} \quad \text{acceptance criterion: } \text{VE} < 10^{-3} \quad (29)$$

→ `scf/diagnostics.py :: virial_error()`

The 10^{-3} threshold is a project convention (V3 of the plan), not a physical constant — it appears hard-coded as a comparison in `dashboard/pages/1_equilibrium.py` and `3_export.py`, not in `units.py` nor in `diagnostics.py` (no physics module defines this threshold as a named constant — it is a UI/gate decision repeated in two pages).

Magnetic virial identity. Exact, and it is the test that links the magnetic sector to the gravitational one:

$$\int \rho \nabla M(u) \cdot \mathbf{r} dV = \int \frac{B^2}{8\pi} dV \quad (30)$$

This identity **does not correspond to any function in the code**. It was verified numerically in an ad hoc way during the debugging of the §1.4 bug (comparing $k_0 \int \rho r \partial u / \partial r dV$ with $\int B^2 / 8\pi dV$ for a converged solution), but there is no function in `scf/diagnostics.py` that computes it nor a committed test that exercises it. Recorded in §7 as a gap.

Mandatory conceptual note

$M(u)$ **is not** the magnetic energy. It is the specific potential of the Lorentz force — only the part that enters the Bernoulli equation. There is no *local* identity between $M(u)$ and $B^2/8\pi$. The *global* identity above is what forces the two quantities to agree in order of magnitude. Measured in this project, in the linear regime (perturbative field), the ratio $(E_{mag}/|W|)/(M_u/H_c)$ is ≈ 0.5 and is constant in k_0 (confirmed by doubling k_0 : both ratios quadruple, the ratio between them does not change); it drifts to ~ 0.38 as the field stops being a small perturbation ($k_0 \gtrsim 10^{-12}$ in the $\rho_c = 10^9$, $R \approx 3 \times 10^8$ cm configuration). This was the observation that, together with the Ampère test, confirmed that the bug in the §1.4 box was real and not a units artifact.

Physical limit. $\mathcal{M} < |W|$ is a rigid constraint: $E_{mag}/|W| \geq 1$ is an impossible configuration (it would violate the virial with $\Pi \geq 0$). Use it as a sanity anchor when reading any result — if the dashboard shows $E_{mag}/|W|$ above a few tenths, be suspicious before believing it.

1.8 The two Bt/Bp ratios

$$\left(\frac{B_t}{B_p}\right)_{\text{energy}} = \frac{E_{tor}}{E_{pol}} \quad (31)$$

$$\left(\frac{B_t}{B_p}\right)_{\text{amplitude}} = \frac{\max |B_\varphi|}{\max |B_{pol}|} \quad (32)$$

→ `scf/toroidal.py :: bt_bp_ratios()`

They differ by orders of magnitude because the toroidal field is confined to a small volume (§1.9). The literature frequently does not say which one it uses. **The dashboard always shows both, labeled** (Tab 1 and Tab 3).

1.9 The toroidal field and the twisted torus

In the barotropic formulation, $\beta = \beta(u)$: the toroidal function depends only on the flux function. It is this condition that cancels the φ component of the Lorentz force and allows the Bernoulli integral to exist (§1.5).

Geometric consequence: outside the star there is no current, so $B_\varphi = 0$ there; since $\beta = \beta(u)$, it must vanish on every flux line that escapes the surface. **The toroidal field is automatically confined to the region of closed poloidal lines** — the twisted torus is not imposed by geometric decree, it falls out of the consistency of the equation.

Adopted functional form:

$$\beta(u) = \zeta (u - u_c)^{m+1} \Theta(u - u_c) , \quad m \geq 1 \quad (33)$$

with u_c the value of u on the last closed line (here, u_c = the value of u at the surface — see the notation distinction in §1.5). The exponent ≥ 1 keeps $\beta\beta'$ continuous at the edge of the torus.

→ `scf/toroidal.py :: impose_toroidal()`

implements this form already substituted into $B_\varphi = \beta/\varpi$: $B_\varphi = \zeta(u - u_c)^{m+1}/\varpi$ for $u > u_c$, 0 outside. `find_uc()` implements the search for u_c as the **maximum of u along the entire stellar surface** ($H = 0$, sweeping θ from pole to equator) — it is this specific choice of “last

closed line” that the code uses; contours with u larger than that maximum, by definition, do not touch the surface anywhere and remain entirely interior.

ζ is not a direct user input parameter — the dashboard asks for the target B_t/B_p (energy) ratio and solves for ζ to reach it, exploiting the fact that B_φ is linear in ζ (so E_{tor} is quadratic):

→ `scf/toroidal.py :: solve_zeta_for_energy_ratio()`

Why the toroidal field is imposed rather than solved for

The condition $\beta = \beta(u)$ confines the toroidal field to a small-volume torus, and the resulting energy ratio — if β were extracted from a general barotropic closure condition instead of chosen freely — comes out to a few percent. **It is not possible to reach $B_t/B_p \sim 1/2$ that way.** That’s why the project solves the barotropic SCF only for the poloidal field (§1.4–§1.6, with $f(u) = k_0$) and imposes the toroidal field on top, at the desired ratio (ζ solved for the target), loading it into Castro out of exact equilibrium and relaxing it with damping (sponge, Tab 3). For a dynamical study, exact equilibrium is not required — it just needs to be close enough that the transient doesn’t destroy the topology.

Torus volume fraction (how much of the star has $u > u_c$):

→ `scf/toroidal.py :: closed_torus_volume_fraction()`

Torus radial extent (used to check mesh resolution, Tab 3):

→ `scf/toroidal.py :: torus_radial_extent()`

, measured along the equator by default.

1.10 Time scales and units

$$t_{dyn} = \sqrt{\frac{R_{eq}^3}{GM}} \quad (34)$$

$$v_A = \frac{\langle B \rangle}{\sqrt{4\pi\bar{\rho}}} \quad (35)$$

$$t_{Alfven} = \frac{R_{eq}}{v_A} \quad (36)$$

→ `dashboard/units.py :: dynamical_time(), alfven_speed(), alfven_time()`

$\langle B \rangle$ and $\bar{\rho}$ (volume averages) are computed in `dashboard/pages/3_export.py` directly with `diagnostics.volume_integral()` — there is no dedicated `mean_field()` function in `scf/`.

The ratio t_{Alfven}/t_{dyn} measures the cost of the simulation: in the weak-field regime it reaches 10^3 – 10^4 , which is prohibitive; in the strong-field regime of this project, with $E_{mag}/|W| \sim 0.1$, it stays at 2–3 (D4 of the plan).

Natural field unit. From $B^2/8\pi \sim G\rho^2 R^2$:

$$B_{unit} = R\rho\sqrt{8\pi G} \quad (37)$$

For $\rho_c \sim 10^9$ and $R \sim 10^8$ cm this gives $\sim 10^{14}$ G, which is also the order of a white dwarf’s virial field. **This formula is not implemented anywhere in the code** — it was used only as an order-of-magnitude estimate during the debugging of the §1.4 bug, to check whether the reported field made physical sense before looking for the numerical error. **Sanity anchor:**

with $E_{mag}/|W| \sim 0.1$, the field shown by the dashboard should be in the 10^{13} – 10^{14} G range; if it comes out very different from that, be suspicious before believing it (it was exactly this suspicion that found the bug).

Castro convention: the field is loaded as $B' = B/\sqrt{4\pi}$. The dashboard **always displays gauss** and converts only at export time (Tab 3).

→ `dashboard/units.py :: gauss_to_castro(), castro_to_gauss()`

Checked in this work cycle: the factor matches the plan ($\sqrt{4\pi} \approx 3.5449$). **Status note:** `gauss_to_castro()` exists and is correct, but **it is not called anywhere in the current export pipeline** (`dashboard/pages/3_export.py` writes `B_phi` to the HDF5 file directly in gauss, with an explicit `units` attribute documenting this). Converting to the Castro B' convention is the responsibility of Castro’s `problem_initialize.H` (not yet written — Phase 0 of the plan is pending), which will read the HDF5 file in gauss and apply `gauss_to_castro()` (or the C++ equivalent) when assembling the internal state. See §7.

All display-unit conversions (gauss, km) — both the number and the string formatting — live in `dashboard/units.py`, the single source of truth (dashboard rule R4): `cm_to_km()`, `g_to_msun()`, `format_gauss()`, `format_km()`, `format_km_value()`.

2 Tab 1 — Equilibrium

→ `dashboard/pages/1_equilibrium.py`

Runs a single SCF pass (§1.6) and shows the result. Uses the entire theoretical core (§1.1–1.10).

Input parameters (sidebar): ρ_c , μ_e , k_0 (optional, toggles the poloidal field on/off), target B_t/B_p ratio and m (toroidal, optional), plus the numerical mesh parameters (N_r , N_θ , l_{max} , tol , max_iter). θ always covers the full $[0, \pi]$ — no equatorial symmetry, a plan decision (D3) so as not to mask asymmetric modes ($m = 1$).

k_0 range. Not known a priori (it depends on ρ_c , R , μ_e in a non-trivial way — see the §1.4 bug, which for a long time made the apparent range look wrong by 3–4 orders of magnitude). The dashboard probes empirically by raising k_0 geometrically until $VE > 10^{-3}$ or the SCF stops converging, on a coarse mesh, and stores the result in a cache in `dashboard/k0_range_cache.json`, indexed by (ρ_c, μ_e) .

→ `dashboard/pages/1_equilibrium.py :: _estimate_k0_max()`

Displayed scalars (“Scalars” table):

Scalar on screen	Definition	Function
M/M_{sun}	total mass / M_{SUN}	<code>scf.total_mass()</code> , <code>units.M_SUN</code>
R_{eq} , R_{pol} (km)	radius where $\rho \rightarrow 0$, at the equator and the pole	<code>diagnostics.equatorial_polar_radii()</code>
R_{pol}/R_{eq}	flattening	direct ratio
ρ_c confirmed	$\rho[r=0]$ post-convergence	should match the input ρ_c
mean ρ	M/volume of the ellipsoid	geometric approximation, not an exact integral
W , $E_{int}=\Pi$, E_{mag} , E_{pol} , E_{tor}	$(R_{eq}, R_{eq}, R_{pol})$ §1.7	<code>diagnostics.virial_error()</code> , <code>magnetic_energies()</code>
$E_{mag}/ W $	dimensionless field strength	direct ratio (sanity anchor, §1.7/1.10)
$B_{pol,max}$, $B_{central}$, $B_{tor,max}$	field values, gauss	maxima/pointwise values of B_r , B_θ , B_ϕ
torus volume fraction	§1.9	<code>toroidal.closed_torus_volume_fraction()</code>
VE	§1.7	<code>diagnostics.virial_error()</code>

Convergence: two plots ($\max |\Delta\rho|/\rho_c$ and VE, both vs. iteration, log scale)

→ `plots.plot_convergence()`, `plots.plot_virial_history()`

The VE history only exists if `hachisu_scf(..., track_virial=True)` — it costs extra integrals every iteration, so it is optional (see §1.6).

Bt/Bp: the two ratios of §1.8, side by side, always labeled.

Meridional-plane figures (§1.3):

- **density:** color map of ρ , with the $H = 0$ boundary (the stellar surface) overlaid in cyan

→ `plots.plot_density()`

- **poloidal field lines:** contours of u — physically, each contour *is* a poloidal field line (§1.3), because $\mathbf{B}$ is tangent to the curves of constant u by construction ($\mathbf{B} \cdot \nabla u = 0$ follows directly from the formulas for B_r, B_θ in terms of derivatives of u). The last closed line (u_c , §1.9) is highlighted in red when a toroidal field is imposed

→ `plots.plot_flux_contours()`

- **toroidal field:** color map of B_φ , shows the confined torus

→ `plots.plot_toroidal()`

All radial axes in km, field always in gauss with a colorbar in scientific notation (rule R4, §1.10).

3 Tab 2 — Sweep

→ `dashboard/pages/2_sweep.py`, `dashboard/sweep_worker.py`

Runs the SCF (§1.6) on a (ρ_c, k_0) grid in parallel (`ProcessPoolExecutor`), with caching by parameter hash (`store.run_exists()/store.save_run()`, §5). Each grid point is an independent call to `scf.hachisu_scf()` followed by the same diagnostics as Tab 1 — no new physics, just orchestration.

→ `dashboard/sweep_worker.py :: run_one()`

— the picklable function each grid worker process runs.

What an equilibrium sequence is. Fixing ρ_c and varying k_0 (or vice versa) produces a sequence of equilibrium configurations. **The sequence ends** when the SCF stops converging or when VE exceeds the acceptance threshold (§1.7) and does not improve with resolution — see the measured numbers in §6. A sequence ending is, in itself, a physical result (not an error to be fixed): it signals the limit of validity of that family of equilibria.

M-R diagram. R_{eq} (km) on the x-axis, $M/M_\odot$ on the y-axis, colored by k_0 . The horizontal line at $1.44 M_\odot$ marks the Chandrasekhar limit **without a field** ($\mu_e = 2$) — it is not the physical limit of the magnetized sequence, it's just a reading reference. Optional overlay of literature data points from `dashboard/data/references/bera_bhattacharya_2014.csv`, if the file exists (it does not currently exist in this repository — no data has been digitized; the dashboard works without it and warns that it's missing).

Why the relevant mass maximum is over the whole plane, not a slice

This is the point that most often causes an incorrect comparison with the literature. $M_{max}(k_0=0)$ already depends on ρ_c : it only approaches the Chandrasekhar limit asymptotically, for high ρ_c ($\sim 10^{11}$ – 10^{12} g/cm³ — see `scf/tests/test_scf_v1.py`, validated to 0.78% in that range). A slice at low ρ_c (for example $\rho_c = 10^9$, used during this project's debugging) gives $M(k_0=0) = 1.39 M_\odot$ — **already below** the field-free Chandrasekhar limit, and any mass measured along that slice (with or without a field) is not comparable to the literature reference number ($M_{max} \sim 1.9 M_\odot$ in Bera & Bhattacharya 2014), which is the maximum taken over the **entire** (ρ_c, k_0) plane, not over a line with k_0 varying at fixed ρ_c . This tab's sweep is exactly what allows that comparison to be made correctly — by mapping the plane, not a slice.

Grid convergence. Points that fail to converge are recorded (not silently dropped) and shown in a separate expander; the grid's convergence rate is, itself, information about where the family of solutions ends.

VE heat map. Over the (ρ_c, k_0) grid, in $\log_{10}(\text{VE})$ — reveals where the method is at the edge of validity without having to read numbers one by one.

4 Tab 3 — Export

→ `dashboard/pages/3_export.py`

Takes an already-saved equilibrium (Tab 1 or 2), imposes the toroidal field (§1.9) and generates the three output artifacts: an HDF5 initial data file, Castro’s inputs, and `run_manifest.json`.

Toroidal imposition. Same function as §1.9 (`toroidal.solve_zeta_for_energy_ratio()`), with this tab’s own controls (the target ratio may differ from the one used when the equilibrium was saved). Continuity of $\beta\beta'$ at the torus edge is **guaranteed analytically** for $m \geq 1$ (it is not recomputed numerically here — it is a property of the functional form of §1.9, $(u - u_c)^{m+1}$ and its derivative go to zero at $u = u_c$ for any $m \geq 1$); the page only confirms that `m_tor` ≥ 1 was respected.

Why the field is initialized from the vector potential, not from $\mathbf{B}$ at cell centers. This is a Castro-side decision (D3 of the plan, section 5), not something the SCF resolves — but the dashboard already prepares the data in that format: the exported HDF5 file contains A_φ (computed as u/ϖ from the converged flux function), **not** B_r , B_θ directly. Castro’s constrained-transport algorithm keeps $\mathbf{B}$ on the mesh faces; initializing via $\nabla \times \mathbf{A}$ interpolated onto the edges guarantees $\nabla \cdot \mathbf{B} = 0$ to machine precision by construction. Initializing with $\mathbf{B}$ values at cell centers does not give that guarantee. **What’s missing:** interpolating A_φ onto the Cartesian mesh edges and computing $\nabla \times \mathbf{A}$ there is the responsibility of Castro’s `problem_initialize_mhd_data.H`, which has not been written yet (Phase 0 of the plan is pending) — the dashboard delivers A_φ on a spherical (r, θ) mesh; interpolation onto Castro’s Cartesian mesh happens on the other side.

$B' = B/\sqrt{4\pi}$ convention. See §1.10. The HDF5 file exported by this tab stores B_φ in pure gauss, with a `units` attribute in the header saying so explicitly — the conversion to the Castro convention does not yet happen in this pipeline (see the status note in §1.10 and the gap in §7).

Derived scales and simulation cost. t_{dyn} , v_A , t_{Alfven} and the $t_{\text{Alfven}}/t_{dyn}$ ratio from §1.10, computed from the $\langle B \rangle$ and $\bar{\rho}$ of the loaded equilibrium. A success badge appears when the ratio is between 0.3 and 3 — the strong-field regime that makes the simulation cheap (D4 of the plan).

Torus resolution check. Given the number of cells per side of Castro’s box (128/256/384) and the box size (a multiple of R_{eq}), computes how many cells cross the torus’s radial extent (`toroidal.torus_radial_extent()`, measured at the equator). Fewer than 10 cells triggers a warning — the torus is small compared to the star and it is the torus that needs to be resolved (plan, section 6).

Box parameters. `castro.small_dens` is chosen as $\rho_c \times 10^{-8}$ — this is a **convention of this dashboard’s design**, not a value derived from physics nor cited in the plan; there is no documented theoretical justification for the 10^{-8} exponent beyond it being a density floor low enough not to perturb the star and high enough not to generate an absurd v_A in the numerical vacuum (the “fluff problem”, plan section 5). The sponge start radius ($1.5 R_{eq}$), the damping duration ($5t_{dyn}$) and the perturbation amplitude ($10^{-4}c_s$, using `eos.sound_speed()` at the center) come directly from the values cited in the plan (section 5), and are not recalculated/optimized by this dashboard.

R5 — export blocked if $VE \geq 10^{-3}$, with no override option. Implemented as a simple `if/else` around the export button — there is no override mechanism at any layer.

5 Tab 4 — Runs

→ `dashboard/pages/4_runs.py`, `dashboard/store.py`

No physics — this module only persists what the other tabs have already computed (dashboard rule R1/R3).

What gets saved, per run, in `dashboard/runs/<hash>/`:

File	Content	Function
<code>params.json</code>	full input parameters	<code>store.save_run()</code>
<code>scalars.json</code>	derived scalars (same as the Tab 1 table)	same
<code>fields.npz</code>	<code>rho</code> , <code>Phi</code> , <code>u</code> , <code>H</code> , <code>Bphi</code> , <code>r</code> , <code>theta</code> on the mesh	same
<code>manifest.json</code>	hash, timestamp, git, dependencies	same

`dashboard/runs/index.csv` aggregates hash + parameters + scalars for all runs, so Tab 4 can load quickly without opening every directory (`store.load_index()`).

Why the git hash matters. `manifest.json` records `git_commit_hash(REPO_ROOT)` — the commit of `scf/` and `dashboard/` (the same git repository covers both, initialized specifically to give this dashboard provenance; `amrex/`, `castro/`, `microphysics/` are excluded via `.gitignore`, being their own repositories). A scalar without its associated commit cannot be reproduced with confidence — if the code changes (for example, the §1.4 bug being fixed), results saved before and after **are not comparable**, even with the same input parameters. `git_dirty()` is also recorded — it flags whether there were uncommitted changes at the time of the run, which makes exact reproduction impossible even knowing the hash.

→ `dashboard/store.py :: git_commit_hash(), git_dirty(), dependency_versions()`

(versions of `numpy`, `scipy`, `streamlit`, `plotly`, `h5py`, `python`).

Cache. The hash is `sha256(json(params, sort_keys=True))[:12]` (`store.params_hash()`) — deterministic in the parameters, used both to name the run’s directory and for Tab 2 to skip already-computed points.

Schema version: there is no `schema_version` field (or equivalent) in `manifest.json` or `scalars.json` currently. When the set of scalars changed during this project (for example, when `B_pol,max` (G) was added to Tab 2’s schema), runs saved before the change ended up with missing columns in `index.csv`, breaking charts that expected the new column — this actually happened during development and was worked around by manually deleting the old cache, not by a versioning mechanism. Recorded in §7 as a gap.

Features: filterable/sortable table (`st.data_editor`, formatted per column per rule R4 — gauss in scientific notation, km with 2 decimal places), side-by-side comparison of two runs, reload a run into Tab 1 (via `st.session_state["reload_run_params"] + st.switch_page`), mark as reference (`store.mark_reference()` — today it only records the flag; no other tab reads reference yet, see §7).

6 Known limitations

Measured numbers, not vague descriptions — all in the $\rho_c = 10^9$ g/cm³, $R \approx 3 \times 10^8$ cm, $nr=161$, $ntheta=65$, $l_{max} = 16$ configuration unless otherwise noted:

- **The $VE < 10^{-3}$ criterion fails above $k_0 \approx 2 \times 10^{-12}$** in this configuration. The last valid point is $k_0 \approx 1.6 \times 10^{-12}$, $M \approx 1.50 M_\odot$, $VE \approx 6.6 \times 10^{-4}$. At $k_0 \approx 2.3 \times 10^{-12}$ ($M \approx 2.02 M_\odot$), $VE \approx 1.57 \times 10^{-3}$ — above the threshold.
- **At that same point ($k_0 \approx 2.3 \times 10^{-12}$) the density peak migrates away from the center** — from $r_idx=1$ (essentially the origin) to $r_idx \approx 25$ (a real radius, not a grid artifact) — and R_{pol}/R_{eq} drops to ~ 0.61 : genuine equatorial evacuation. The anchor at $\rho_c(r=0)$ (§1.5) loses physical meaning right there, because the center is no longer the point of maximum density.
- **Whether the VE failure above is insufficient resolution or a genuine sequence termination is an open question — but there is strong evidence for the second hypothesis:** a convergence study done in this project (l_{max} from 16 to 48, mesh from 129^2 to 385^2 , nearly $9\times$ more points) kept VE at $1.2\text{--}1.6 \times 10^{-3}$, **with no downward trend** — a plateau above the threshold, not a curve converging to zero. This is the expected signature of a physical termination, not of under-resolution.
- **All the sequence results (k_0 varying) documented above are from a single $\rho_c = 10^9$ slice**, where $M(k_0=0) = 1.39 M_\odot$ — below the field-free Chandrasekhar limit itself ($1.44 M_\odot$). Comparisons with literature mass maxima ($M_{max} \sim 1.9 M_\odot$, Bera & Bhattacharya 2014) require sweeping the entire (ρ_c, k_0) plane (Tab 2), not a slice — see §3.
- **Castro’s $B' = B/\sqrt{4\pi}$ conversion is not applied anywhere in the current export pipeline** (§1.10, §4) — the exported HDF5 file stores B in pure gauss, documented via an attribute. This is left as the responsibility of Castro’s `problem_initialize.H` (not yet written).
- **Phase 0 of the plan (building Castro with `USE_MHD=TRUE`) is pending** — system dependencies (`gfortran`, `libhdf5-openmpi-dev`, `libopenmpi-dev`) have not yet been installed in this environment. Nothing in Tab 3 has been tested against a real Castro build.

7 Open questions

Gaps identified while writing this document — not filled in on our own initiative (rule G1):

1. **Flux consistency test** ($u(\varpi, z) = \int_0^\varpi B_z \varpi' d\varpi'$, §1.4) was proposed during debugging of the Green’s-function bug but never implemented as a separate test. The Ampère test alone was enough to find and confirm the bug; the flux test would serve as a second, independent line of defense.
2. **Magnetic virial identity** ($\int \rho \nabla M(u) \cdot \mathbf{r} dV = \int B^2/8\pi dV$, §1.7) has no dedicated function in `diagnostics.py` and no committed test — it was only verified numerically in an ad hoc way during one debugging session.
3. **The formula $J_\varphi = c\rho\varpi f(u)$** (§1.4) does not correspond to any function — it is never computed as a named quantity in the code.
4. $B_{unit} = R\rho\sqrt{8\pi G}$ (§1.10, natural field unit) is not implemented — it was used only as an order-of-magnitude estimate during debugging.
5. **No `schema_version`** in `manifest.json/scalars.json` (§5) — changes to the set of scalars have already broken `index.csv` for old runs once during this project.
6. **`store.mark_reference()`** records the reference flag in the index, but no other tab (in partic-

ular Tab 2, which per the original prompt should show reference runs on its charts) reads that flag yet.

7. **castro.small_dens** = $\rho_c \times 10^{-8}$ (§4) is a convention with no documented theoretical justification — it works as a starting point, not as a calibrated value.
8. **The original question that motivated the whole §1.4 bug investigation** — whether the ratio $(E_{mag}/|W|)/(M_u/H_c) \approx 0.5$ (linear regime) has a closed-form derivation, or is just what is observed numerically for this EOS and this parameter range — was not resolved analytically, only confirmed as self-consistent (constant under $k_0 \rightarrow 2k_0$).
9. **gauss_to_castro()/castro_to_gauss()** exist and are correct (checked in this work cycle), but are not called by any export code yet — see the limitation in §6.

8 References

SCF method and magnetized equilibria

- Hachisu, I. 1986, ApJS 61, 479 — the SCF method
- Tomimura, Y. & Eriguchi, Y. 2005, MNRAS — twisted torus, canonical reference
- Lander, S. K. & Jones, D. I. 2009, MNRAS — mixed fields, free functions
- Lander, S. K. & Jones, D. I. 2012 — stability of mixed fields

Magnetized white dwarfs

- Das, U. & Mukhopadhyay, B. 2014, MNRAS 445, 3951 — SCF for magnetized WDs
- Bera, P. & Bhattacharya, D. 2014, MNRAS — self-consistent M-R with Lorentz force
- Bera, P. & Bhattacharya, D. 2016, MNRAS 456, 3375 — field geometry
- Bera, P. & Bhattacharya, D. 2017, MNRAS 465, 4026 — perturbation study
- Nityananda, R. & Konar, S. 2014 — critique of super-Chandrasekhar models
- Coelho, J. G. et al. 2014 — virial limits

Stability

- Markey, P. & Tayler, R. J. 1973 — the $m = 1$ instability
- Braithwaite, J. & Spruit, H. C. 2004 — relaxation to a stable configuration
- Braithwaite, J. & Nordlund, Å. 2006

Codes

- Castro: <https://github.com/AMReX-Astro/Castro>
- MHD documentation: <https://amrex-astro.github.io/Castro/docs/mhd.html>
- XNS: <https://www.arcetri.inaf.it/science/ahead/XNS/>